

Claude Code Just Cut MCP Context Bloat by 46.9% — 51K Tokens Down to 8.5K With Tool Search

Joe Njenga : 26-33 minutes : 1/16/2026



Member-only story



18 min read

Jan 16, 2026

Press enter or click to view image in full size



I watched my context window die before I could write a single prompt — 67,000 tokens gone, ***just from connecting four MCP servers to Claude Code.***

That's what one Reddit user documented, and it's not an isolated case. MCP servers have become wildly popular since Anthropic introduced the Model Context Protocol, but there's been a massive problem hiding underneath all that excitement.

If you are not a Medium paid member, [you can read the story here for FREE.](#)

Every tool from every connected server gets preloaded into your model's context window, before you type anything.

Tool names, descriptions, full JSON schemas, parameters, types, constraints.

Multiply that by 50, 100, or 167 tools and you're burning through tokens faster than you can say "hello, Claude."

The Claude Code team just fixed this with Tool Search, and it's already showing a 46.9% reduction in total agent tokens during actual use.

This changes the economics of AI coding with MCP.

We have all become dependent on MCP servers, and it seems this has been a huge problem, especially for Claude Code users.

Scale of the Problem

MCP servers are everywhere now: GitHub, Docker, Notion, Slack, Salesforce.

The protocol works well for connecting AI agents to external tools and data sources, which is why adoption has exploded over the past few months.

But what happens when you connect multiple servers at the same time?

- Each server exposes dozens of tools.
- Each tool needs a complete definition in context, and the definition includes everything the model needs to know about how to use that tool.

A single well-documented tool can easily consume 400–800 tokens.

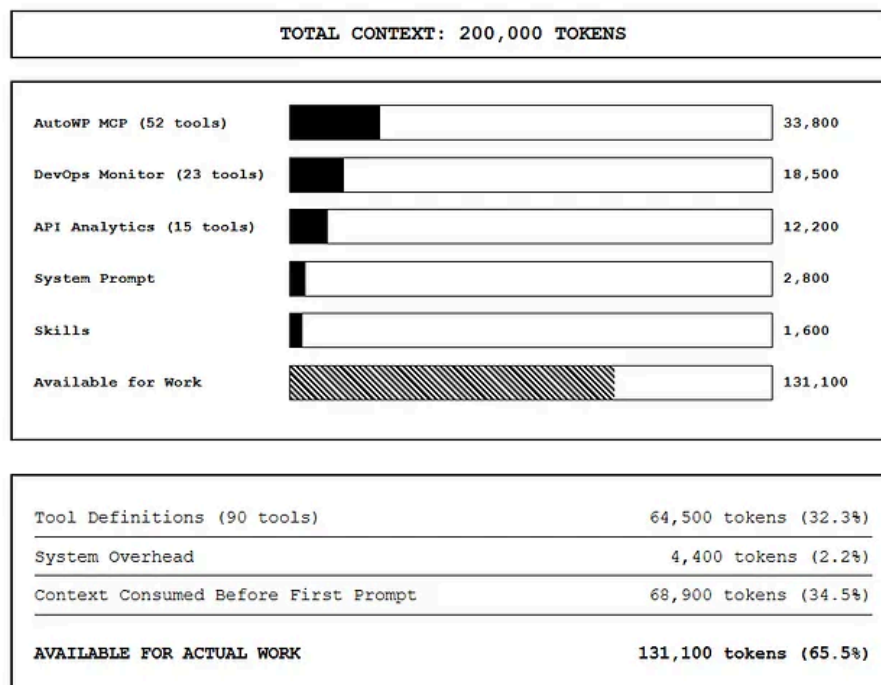
Multiply that across just 50 tools and you're looking at 20,000–40,000 tokens minimum.

One setup using 167 tools across four servers can consume over 60,000 tokens.

That's nearly half of Claude Opus 4's 200K context window, and you haven't even started your actual task yet.

This is an example of a 200K context breakdown:

Press enter or click to view image in full size



It gets worse when you factor in that most developers don't just use MCP.

They also have skills, custom instructions, and conversation history competing for that same context space.

Models perform worse when you stuff too much into their context window.

It's not just about hitting token limits; it's about the quality of responses degrading as context gets cluttered.

The approach of preloading everything made sense when we had 3–5 tools.

But,

MCP has moved way beyond that point, and the tooling needed to catch up.

Claude Code team has solved this problem with Tool Search, and both Cursor and Anthropic have now implemented this approach with measurable results.

I've been [writing about MCP since Anthropic released it](#), [built multiple MCP projects](#), and have been teaching developers [how to use Claude Code](#) for nearly a year now.

I've seen firsthand how context bloat kills otherwise solid implementations.

I also recently shared this approach: [Code Execution With MCP- Anthropic Just Solved AI Agent Bloat — 150K Tokens Down to 2K](#)

But now,

The new **Advanced Tool Use on Claude** takes this to the next level by cutting the context bloat further.

I will walk you through using examples of MCP projects I have previously built to show you how this makes a difference. I will also follow up with my usual Claude Code practical testing tutorial and implementation to help you learn more. If you missed out on the other articles check out the list — [Claude Code Tutorials](#)

Understanding the Context Bloat Problem

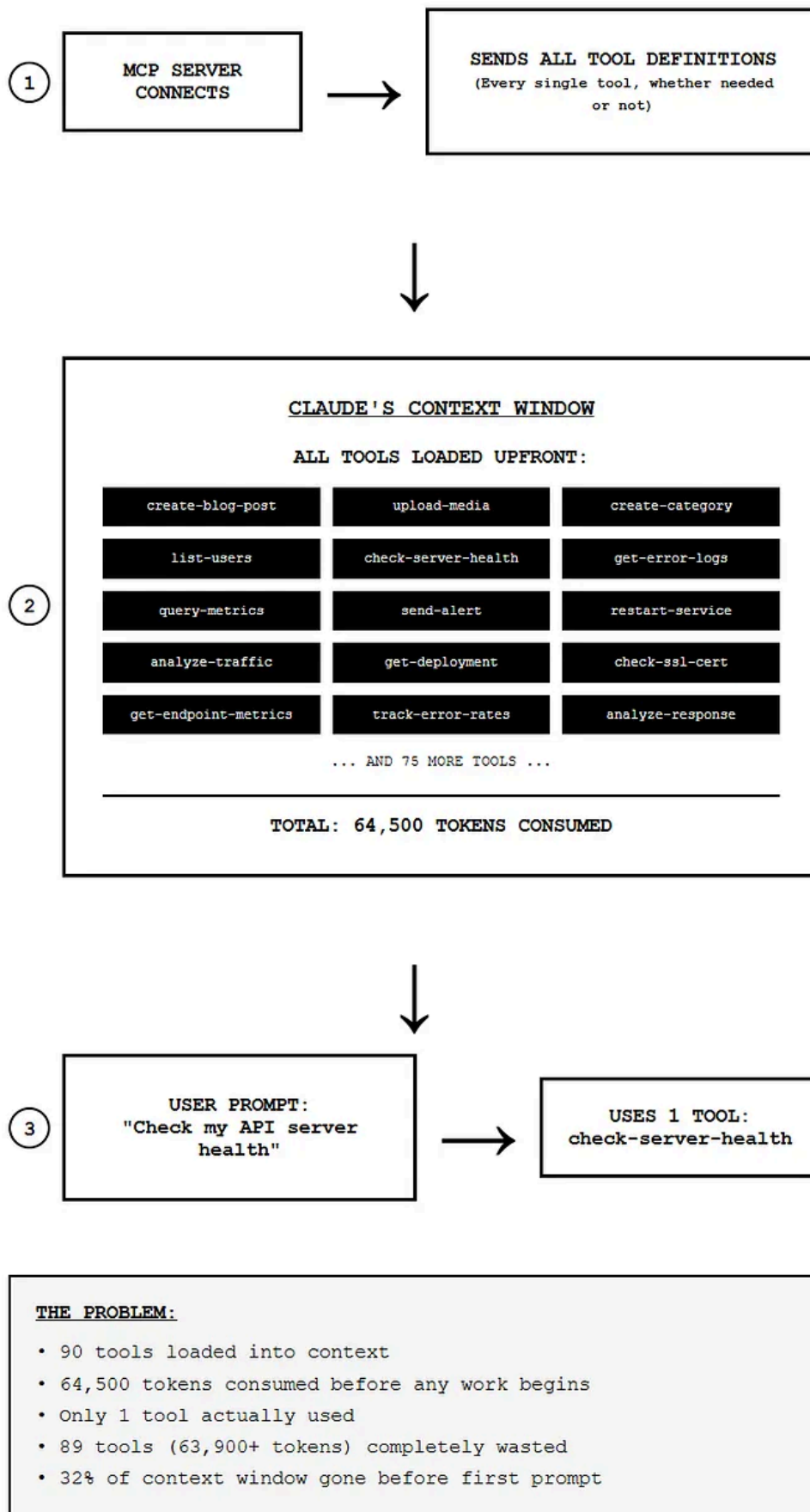
Every connected MCP server exposes its tools to Claude through detailed definitions.

These definitions tell the model everything: what the tool does, what parameters it accepts, their types, required vs optional fields, constraints, and return formats.

The model needs to know what tools are available, but when you scale to dozens or hundreds of tools, this approach breaks down fast.

How Traditional MCP Loading Works

Press enter or click to view image in full size



When you connect an MCP server to Claude Code, the server sends complete tool definitions for every available tool.

Claude loads all of these into its context window before you write your first prompt. The model now “knows” about every tool, whether it needs them or not.

Let me show you what a single tool definition looks like.

Here's a code sample from my AutoWP MCP server that I created back in May:

```
{
  "name": "create-blog-post",
  "description": "Create and publish blog posts to WordPress sites. Supports setting title, content, categories, tags, featured images, and post status (draft, published, scheduled). Handles content formatting and metadata assignment.",
  "input_schema": {
    "type": "object",
    "properties": {
      "title": {
        "type": "string",
        "description": "Post title"
      },
      "content": {
        "type": "string",
        "description": "Post content in HTML or markdown format"
      },
      "status": {
        "type": "string",
        "enum": ["draft", "publish", "future"],
        "description": "Publication status"
      },
      "categories": {
        "type": "array",
        "items": {"type": "integer"},
        "description": "Category IDs to assign"
      },
      "tags": {
        "type": "array",
        "items": {"type": "integer"},
        "description": "Tag IDs to assign"
      },
      "featured_media": {
        "type": "integer",
        "description": "Featured image media ID"
      },
      "excerpt": {
        "type": "string",
        "description": "Post excerpt"
      }
    },
    "required": ["title", "content"]
  }
}
```

That single tool definition consumes roughly 400 tokens, then multiply that by the 52 tools in my AutoWP MCP server.

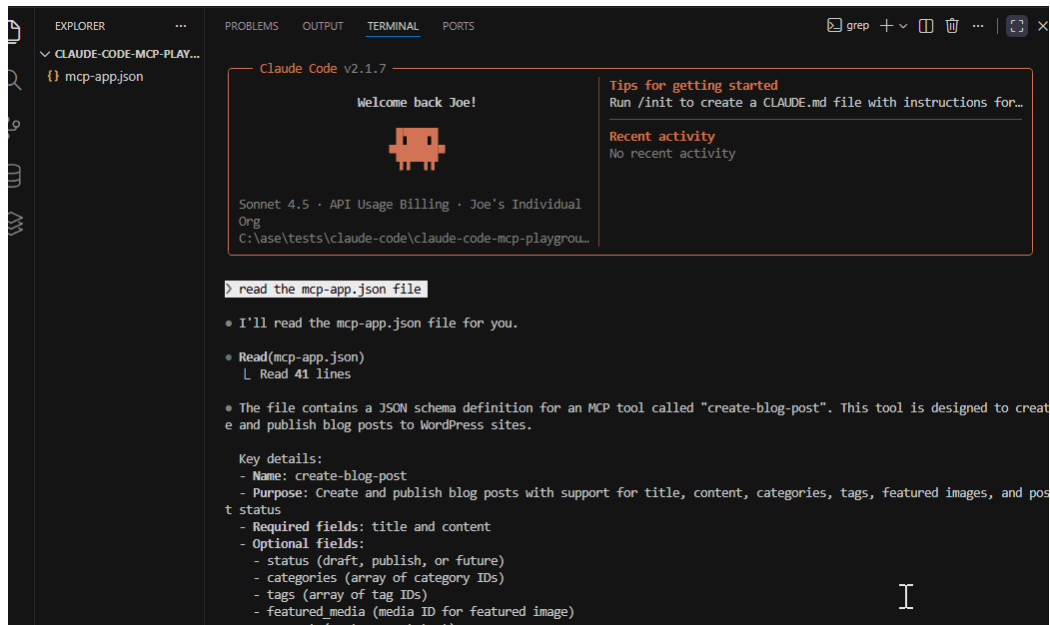
That's approximately **20,800** tokens just for all the automation tools.

But do my numbers make sense?

I know it's easier to throw up numbers and make it look sensible, let's get down to the real test.

I placed the code above in a JSON file and asked Claude Code to read it :

Press enter or click to view image in full size



Before you get misled by the **/cost** figures, which were :

> /cost

```
└─ Total cost:                $0.1122
   Total duration (API):      28s
   Total duration (wall):    2m 37s
   Total code changes:       0 lines added, 0 lines removed
   Usage by model:
       claude-haiku:  125 input, 25 output, 0 cache read, 0
cache write ($0.0003)
       claude-sonnet:  30 input, 1.0k output, 41.3k cache
read, 22.5k cache write ($0.1120)
```

I did a follow-up question since the 30 input tokens you see in Claude Code's cost breakdown is for processing your command, NOT the size of the tool definition itself.

Press enter or click to view image in full size

```
The content field accepts HTML or markdown format, and the status field controls whether posts are saved as drafts, published immediately, or scheduled for future publication.

> /cost
{
  "Total cost": "$0.1122",
  "Total duration (API)": "28s",
  "Total duration (wall)": "2m 37s",
  "Total code changes": "0 lines added, 0 lines removed",
  "Usage by model": {
    "claude-haiku": "125 input, 25 output, 0 cache read, 0 cache write ($0.0003)",
    "claude-sonnet": "30 input, 1.0k output, 41.3k cache read, 22.5k cache write ($0.1120)"
  }
}

> How many tokens is this file?
```

This was the response :

- Based on the file I just read, the mcp-app.json file is approximately 200–250 tokens.

The file contains 41 lines of JSON with:

- A schema definition structure
- Several descriptive text strings
- Property definitions and enumerations
- About 800–900 characters total

The exact token count depends on the tokenizer being used (different models use different tokenizers), but for Claude's tokenizer, this small JSON schema file would be in that 200–250 token range.

Now, when MCP registers this tool with Claude, it adds protocol formatting, metadata, and server context — roughly doubling the bare JSON size. So that 200–250 token definition becomes 400–500 tokens in actual context consumption.

With Tool Use Examples enabled (which improves accuracy), it can reach 600–700 tokens per tool.

Back in May, when [I was writing extensively about MCP here on Medium](#), I created [AutoWP as a hobby project to automate my WordPress publishing workflow](#).

It slowly turned into a production-useful server that I now use for my side projects, where I need to often manage content across multiple WordPress sites.

The server handles everything from creating blog posts to managing media, users, categories, and even plugin management.

You can check it out on GitHub at [Autowpnmcp](#).

While AutoWP MCP demonstrates the scale of the problem with its 52 tools, I want to focus the rest of this article on my other ***DevOps Monitor MCP, which is more relatable to most developers.***

The DevOps Monitor MCP is something I built out of necessity.

When you're running multiple projects, you need constant visibility into server health, error logs, API performance, and deployment status.

I was wasting hours reading through server logs and checking dashboards manually. This MCP server connects to my infrastructure and gives Claude direct access to:

- check-server-health - Monitor CPU, memory, disk usage across servers
- get-error-logs - Pull and analyze application error logs
- query-metrics - Fetch API response times, request counts, error rates
- send-alert - Trigger notifications to Slack or email
- get-deployment-status - Check current deployment versions and health
- restart-service - Restart failed services or containers
- analyze-traffic-patterns - Review traffic spikes and anomalies
- check-ssl-certificates - Verify SSL certificate expiration dates

About 15 more tools for infrastructure monitoring.

Here's what one of these tool definitions looks like:

```
{
  "name": "check-server-health",
  "description": "Monitor server health metrics including CPU usage, memory consumption, disk space, and active processes. Returns current values and threshold warnings for infrastructure monitoring.",
  "input_schema": {
    "type": "object",
    "properties": {
      "server_id": {
        "type": "string",
        "description": "Server identifier or hostname to check"
      },
      "metrics": {
        "type": "array",
        "items": {
          "type": "string",
          "enum": ["cpu", "memory", "disk", "network", "processes"]
        }
      },
      "description": "Specific metrics to retrieve. If not specified, returns all metrics."
    },
    "include_historical": {
```

```

        "type": "boolean",
        "description": "Include last 24 hours of historical data",
        "default": false
    },
    "alert_threshold": {
        "type": "object",
        "properties": {
            "cpu_percent": {"type": "number"},
            "memory_percent": {"type": "number"},
            "disk_percent": {"type": "number"}
        },
        "description": "Custom alert thresholds for this check"
    }
},
"required": ["server_id"]
}
}

```

That's about 580 tokens for a single monitoring tool. Multiply that across all the DevOps tools, and you're looking at serious context consumption.

The MCP saves me time spent monitoring dashboards, log aggregators, and terminal windows.

This is the server we'll use for code examples throughout the article since most developers deal with infrastructure monitoring, not WordPress automation.

Tokens Breakdown

Let's look at how quickly tokens add up when you connect multiple MCP servers.

Say I'm running Claude Code with three of my custom servers:

AutoWP MCP (52 tools for WordPress automation)

- create-blog-post: ~400 tokens
- upload-media: ~720 tokens
- create-category: ~450 tokens
- list-users: ~490 tokens
- set-featured-image: ~540 tokens
- ...47 more tools
- **Total: ~20,800 tokens**

DevOps Monitor MCP (monitoring my side projects)

- check-server-health: ~580 tokens
- get-error-logs: ~650 tokens
- query-metrics: ~720 tokens
- send-alert: ~490 tokens
- get-deployment-status: ~560 tokens
- restart-service: ~520 tokens

- ...and more tools
- **Total: ~18,500 tokens**

API Analytics MCP (tracking API usage)

- get-endpoint-metrics: ~640 tokens
- analyze-response-times: ~710 tokens
- track-error-rates: ~590 tokens
- ...more tools
- **Total: ~12,200 tokens**

That's approximately 51,500 tokens consumed. And this is just with three custom servers that I built for my own workflows.

Cascading Effects

This creates problems that compound:

- **Context window exhaustion** — *Opus 4 has 200K tokens, burn 51K on tool definitions, and you've lost 26% of your available space for the actual conversation, code, and results.*
- **Degraded model performance** — *Models work better with focused context. When you stuff tool definitions into the window, the model has to wade through massive amounts of irrelevant information.*
- **Cost multiplication** — *Every token costs money. If you're running agents at scale, spending 51k tokens per on tool definitions makes no sense.*
- **Tool limitation paradox** — *You want to give your agent access to more tools, but each new tool makes the problem worse.*

The root cause is MCP servers don't know what task you're about to give Claude.

They can't predict which tools you'll need, so they expose everything.

The model can't selectively load tools because it doesn't know what's available until everything is loaded.

This is the problem Tool Search solves, and it does it through a pattern that should sound familiar if you've followed my articles on Agent Skills.

Claude discovers tools when needed, rather than preloading everything.

How Tool Search Works

Tool Search introduces a simple, powerful threshold: if your MCP tool descriptions would consume more than 10% of the context window, Claude switches to dynamic discovery.

For a 200K context model, that's 20K tokens. Below that threshold, MCP works as before. When the tokens go above that threshold, tool Search kicks in.

Dynamic Discovery Mechanism

Claude Code does not load all tool definitions upfront. It now treats tool descriptions as searchable references.

The model receives minimal static context: just the Tool Search Tool itself and your most critical tools.

When Claude needs a specific capability, it searches for relevant tools using natural language, loads 3–5 of the most relevant tools based on your prompt, then uses those tools normally.

Press enter or click to view image in full size

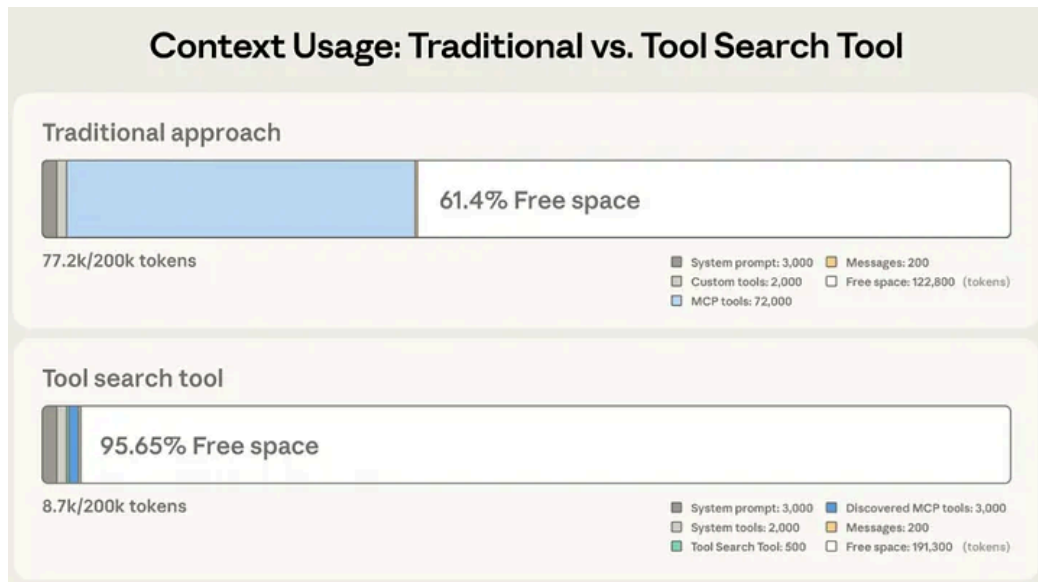


Image Credit — [Anthropic](#)

Let me show you how this works with my DevOps Monitor MCP.

Traditional Approach — Loading Everything

Here's what happened before Tool Search when I connected my DevOps MCP:

```
client.messages.create(  
  model="claude-sonnet-4-20250514",  
  max_tokens=4096,  
  tools=[  
    {  
      "name": "check-server-health",  
      "description": "Monitor server health metrics...",  
      "input_schema": {...}  
    },  
    {  
      "name": "get-error-logs",  
      "description": "Pull and analyze application error  
logs...",  
      "input_schema": {...}  
    },  
    {  

```

```

        "name": "query-metrics",
        "description": "Fetch API response times...",
        "input_schema": {...}
    },

    ],
    messages=[{"role": "user", "content": "Check my API server
health"}]
)

```

Claude loads all 23 DevOps tools, consuming 18,500 tokens, then uses exactly 1 tool (check-server-health) to answer my question.

| The other 22 tools are a complete waste of context.

Tool Search Approach — Dynamic Loading

Here's the same setup with Tool Search enabled:

```

client.beta.messages.create(
    betas=["advanced-tool-use-2025-11-20"],
    model="claude-sonnet-4-5-20250929",
    max_tokens=4096,
    tools=[

        {
            "type": "tool_search_tool_regex_20251119",
            "name": "tool_search"
        },

        {
            "type": "mcp_toolset",
            "mcp_server_name": "devops-monitor",
            "default_config": {"defer_loading": true},
            "configs": {

                "send-alert": {
                    "defer_loading": false
                }
            }
        }
    ],
    messages=[{"role": "user", "content": "Check my API server
health"}]
)

```

What happens now:

1. **Initial load:** Only Tool Search Tool (~500 tokens) + send-alert (~490 tokens) = ~990 tokens
2. **Claude searches:** "server health monitoring tools"
3. **Tool Search returns:** check-server-health, query-metrics, get-deployment-status
4. **These 3 tools get loaded:** ~1,890 tokens
5. **Total context used:** ~2,880 tokens

That's an **84% reduction** in tool-related tokens for this specific task.

How the defer_loading Flag Works

The `defer_loading: true` flag tells Claude "this tool exists, but don't load its full definition yet."

For my DevOps MCP, here's how I configure which tools to defer:

```
{
  "type": "mcp_toolset",
  "mcp_server_name": "devops-monitor",
  "default_config": {
    "defer_loading": true
  },
  "configs": {

    "send-alert": {
      "defer_loading": false
    },
    "check-server-health": {
      "defer_loading": false
    },

    "restart-service": {
      "defer_loading": true
    },
    "analyze-traffic-patterns": {
      "defer_loading": true
    },
    "get-deployment-status": {
      "defer_loading": true
    },
    "check-ssl-certificates": {
      "defer_loading": true
    }
  }
}
```

I keep `send-alert` and `check-server-health` always loaded because I use them in almost every monitoring task.

The other 21 tools, I let Claude discover them when needed.

Real Search in Action

Let me show you what happens when I ask Claude to investigate a production issue.

My prompt: *"My API is responding slowly. Figure out what's wrong and alert me if it's critical."*

Behind the scenes:

Step 1: Claude receives the prompt

- Sees: Tool Search Tool + 2 loaded DevOps tools
- Decides: Need to search for performance/API tools

Step 2: Claude calls tool_search

Search query: "API performance metrics slow response"

Returns:

- query-metrics
- analyze-traffic-patterns
- get-error-logs

Step 3: These 3 tools get loaded (~2,100 tokens)

Step 4: Claude uses them

- Calls query-metrics(endpoint="/api/users")
- Calls get-error-logs(severity="error", last_hours=2)
- Analyzes results
- Calls send-alert() (already loaded)

Total tools loaded: 5 out of 23

Token savings: ~13,500 tokens

Claude found what it needed without loading deployment tools, SSL checkers, or service restart capabilities.

Search Methods Available

Anthropic provides two built-in search implementations:

Regex search (simple keyword matching):

```
{"type": "tool_search_tool_regex_20251119", "name": "tool_search"}
```

BM25 search (more sophisticated ranking):

```
{"type": "tool_search_tool_bm25_20251119", "name": "tool_search"}
```

You can also implement a custom search using embeddings or other strategies.

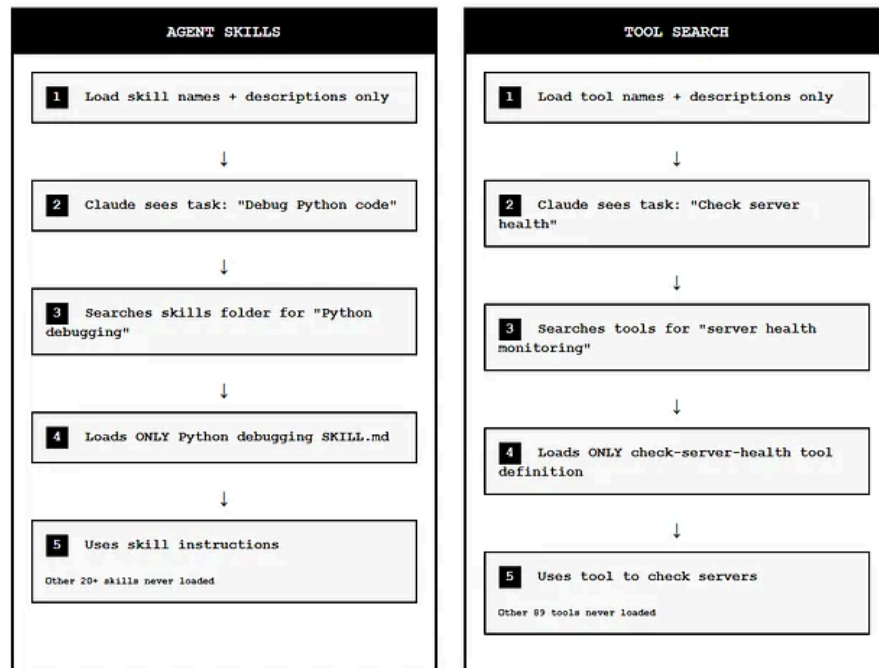
The important part is that the search returns the most relevant tool references, which then get expanded into full definitions.

Connection to Agent Skills

If you've been following my articles on Agent Skills, this pattern should feel familiar.

Agent Skills work the same way: instead of loading all skill documentation into context, Claude sees skill names and descriptions in a folder structure, then loads full skill details only when relevant to the current task.

Press enter or click to view image in full size



Both solve the same problem: how do you give an agent access to extensive capabilities without overwhelming its context window?

The answer in both cases is progressive disclosure through dynamic discovery.

10% Threshold Logic

Why does Tool Search only kick in above 10% of context?

Below that threshold, the overhead of searching outweighs the benefits. If you only have 5–6 tools consuming 3,000 tokens total, just load them all. It's simpler and faster.

But once tool definitions cross 20K tokens (10% of 200K), the math flips:

Without Tool Search:

- 20K+ tokens consumed upfront
- Model processes all tools on every turn
- Higher chance of tool selection errors

With Tool Search:

- ~500 tokens for search tool

- 2–3K tokens for discovered tools
- Focused context = better tool selection

Server Instructions

With Tool Search enabled, the `server_instructions` field in your MCP configuration becomes much more important.

This field helps Claude understand what your server provides without loading every tool; it's like a meta-documentation.

Here's what I use for my DevOps Monitor MCP:

```
{
  "server_instructions": "Provides infrastructure monitoring and
  DevOps automation tools. Capabilities include server health
  monitoring, error log analysis, performance metrics tracking,
  deployment status checks, SSL certificate validation, traffic
  pattern analysis, and automated alerting. Use this server for
  production infrastructure visibility, debugging performance
  issues, and automated operations tasks."
}
```

When Claude sees a task related to “server performance” or “production issues,” it knows to search this server’s tools first.

This is similar to how Agent Skills use SKILL.md files to describe their capabilities. The description guides Claude’s discovery process without consuming tokens for detailed tool definitions.

Implementation & Results

Both Cursor and Anthropic’s Claude Code team have now implemented Tool Search, and the results validate the approach.

46.9% Token Reduction

Anthropic ran A/B tests on actual usage. In runs that called at least one MCP tool, Tool Search reduced total agent tokens by 46.9%.

Press enter or click to view image in full size

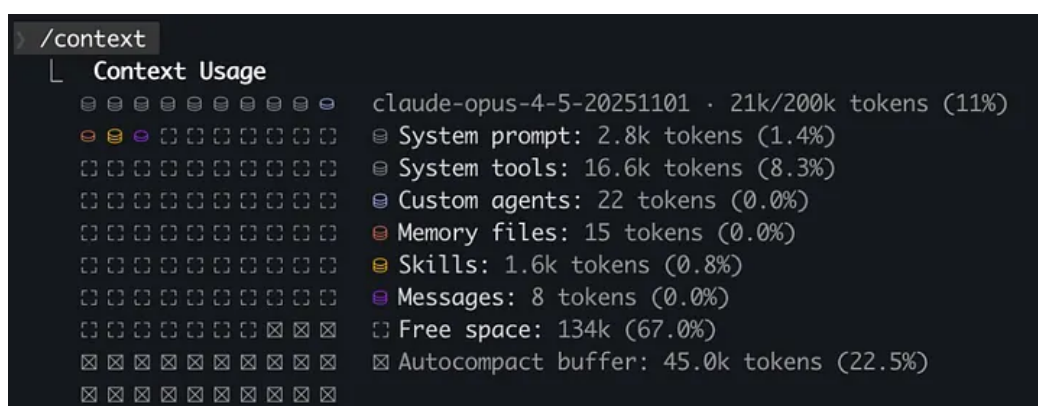


Image Credit — [Thariq](#)

To put this in practical terms, with my DevOps Monitor MCP.

Before Tool Search, a typical monitoring session looked like this:

Context Budget: 200,000 tokens
System prompt: 2,800 tokens
DevOps MCP (23 tools): 18,500 tokens
API Analytics MCP (15 tools): 12,200 tokens
Skills: 1,600 tokens

Overhead: 35,100 tokens (17.5%)
Available for work: 164,900 tokens

Actual tools used: 4
Wasted tool definitions: 34 tools ($\approx 25,000$ tokens)

With Tool Search:

Context Budget: 200,000 tokens
System prompt: 2,800 tokens
Tool Search Tool: 500 tokens
Always-loaded tools (2): 1,070 tokens
Skills: 1,600 tokens

Initial overhead: 5,970 tokens (3%)
Available for work: 194,030 tokens

Tools discovered and loaded: 4 ($\approx 2,600$ tokens)
Total overhead: 8,570 tokens (4.3%)
Savings: 26,530 tokens (75.6% reduction in tool overhead)

That's 26,530 tokens freed up for conversation, code analysis, and results. Over hundreds of sessions per month, this compounds into real cost savings and faster responses.

Implementation Example

Here's how I have updated and set up Tool Search for my production DevOps workflow:

```
import anthropic

client = anthropic.Anthropic(api_key="your-api-key")

response = client.beta.messages.create(
    betas=["advanced-tool-use-2025-11-20"],
    model="claude-sonnet-4-5-20250929",
    max_tokens=4096,

    system="""You are a DevOps assistant with access to
infrastructure
monitoring and automation tools.
```

Available capabilities:

- Server health monitoring (CPU, memory, disk, network)
- Error log analysis and debugging
- API performance metrics and response time tracking
- Deployment status and version checking
- SSL certificate validation
- Automated alerting to Slack/email
- Service restart and recovery operations

When you need specific tools, search for them by describing what

you need. Focus on the task at hand and only load relevant tools."

""

```
tools=[

{
  "type": "tool_search_tool_regex_20251119",
  "name": "tool_search"
},

{
  "type": "mcp_toolset",
  "mcp_server_name": "devops-monitor",
  "default_config": {
    "defer_loading": true
  },
  "configs": {

    "send-alert": {
      "defer_loading": false
    },
    "check-server-health": {
      "defer_loading": false
    },

    "get-error-logs": {"defer_loading": true},
    "query-metrics": {"defer_loading": true},
    "analyze-traffic-patterns": {"defer_loading":
true},

    "get-deployment-status": {"defer_loading": true},
    "restart-service": {"defer_loading": true},
    "check-ssl-certificates": {"defer_loading": true},
    "get-container-logs": {"defer_loading": true},
    "monitor-database-performance": {"defer_loading":
true},

    "track-api-quotas": {"defer_loading": true},
    "verify-backup-status": {"defer_loading": true}
```

```

    },
    {
      "type": "mcp_toolset",
      "mcp_server_name": "api-analytics",
      "default_config": {
        "defer_loading": true
      }
    }
  ],

  messages=[{
    "role": "user",
    "content": "Check if all production services are healthy
and alert me if anything is critical"
  }]
)
print(response.content)

```

This setup gives Claude access to 38+ tools across two MCP servers while only loading 3 tools upfront (Tool Search + 2 critical DevOps tools).

My System Prompt Example

The system prompt is important for guiding Claude's tool discovery.

Here's what I learned works well:

Good system prompt:

You are a DevOps assistant with access to infrastructure monitoring tools.

Available capabilities:

- Server health monitoring
- Error log analysis
- Performance metrics tracking
- Deployment management
- Automated alerting

Search for specific tools when you need them. Focus on the current task.

Bad system prompt:

You have access to many tools. Use them when needed.

The good prompt gives Claude enough context to know what's available without listing every tool. It sets expectations that tools need to be discovered dynamically.

Deferring Entire Servers While Keeping High-Use Tools

One powerful pattern is deferring an entire MCP server but keeping 1–2 frequently-used tools always loaded.

For my DevOps MCP, I defer everything except the tools I use in almost every session:

```
{
  "type": "mcp_toolset",
  "mcp_server_name": "devops-monitor",

  "default_config": {
    "defer_loading": true
  },

  "configs": {
    "send-alert": {
      "defer_loading": false,
      "description": "Always available for critical alerts"
    },
    "check-server-health": {
      "defer_loading": false,
      "description": "Primary monitoring tool, used frequently"
    }
  }
}
```

This gives me instant access to alerting and basic health checks while keeping 21 other tools discoverable on-demand.

Final Thoughts

Tool Search changes how we build with MCP.

Before, connecting multiple MCP servers meant choosing between functionality and context efficiency. Now you can have both.

The new pattern is progressive disclosure through dynamic discovery. It works for Agent Skills, and now it works for MCP tools.

We are still learning the best way to refine this for production, and in the coming days, I will share more lessons learned.

How many MCP servers are you running right now, and how much context are they consuming? Does this help?

Resources

- [Advanced Tool Use on Claude Developer Platform](#) — Anthropic's official guide on Tool Search, Programmatic Tool Calling, and Tool Use Examples

- [Tool Search Tool Documentation](#) — API reference and implementation guide
- [Dynamic Context Discovery](#) — Cursor's blog on their implementation approach
- [Model Context Protocol Specification](#) — MCP protocol documentation
- [Twitter Release Article](#) — Tool Search Now in Claude Code

My Projects & Tutorials:

- [AutoWP MCP Server](#) — WordPress automation with 52 tools
- [Code Execution With MCP — 150K Tokens Down to 2K](#) — My previous article on programmatic tool calling
- [Claude Code Tutorials](#) — Complete guide series on building with Claude Code

Claude Code Course

Every day, I'm working hard to build the ultimate Claude Code course, which demonstrates how to create workflows that coordinate multiple agents for complex development tasks. It's due for release soon.

It will take what you have learned from this article to the next level of complete automation.

New features are added to Claude Code daily, and keeping up is tough.

The course explores Agents, Hooks, advanced workflows, and productivity techniques that many developers may not be aware of.

Once you join, you'll receive all the updates as new features are rolled out.

This course will cover:

- *Advanced subagent patterns and workflows*
- *Production-ready hook configurations*
- *MCP server integrations for external tools*
- *Team collaboration strategies*
- *Enterprise deployment patterns*
- *Real-world case studies from my consulting work*

If you're interested in getting notified when the Claude Code course launches, [click here to join the early access list](#) →

(Currently, I have 3000+ already signed-up developers)

I'll share exclusive previews, early access pricing, and bonus materials with people on the list.